

Informe de Seguridad — OWASP Top 10 (2021)

Sistema MediClick — Plataforma de gestión clínica multi-tenant

Proyecto	MediClick (NestJS + GraphQL + Prisma + PostgreSQL / Next.js)
Alcance	Auditoría y remediación dirigida sobre las 10 categorías OWASP, backend server/
Rama de trabajo (ronda 1)	feat/payments-mp-redirect
Integrado a main (ronda 1)	PR #35 — merge commit e754494 (2026-06-30)
Ronda 2 — cierre de pendientes	rama feat/interop-fase1-proyeccion , commits 882474d y cc7b24c (2026-07-09)
Ronda 3 — corrección de alcance + pruebas	commits 6c91c8d y 0c4fa16 (2026-07-09)
Fecha del informe	30 de junio de 2026 — actualizado 9 de julio de 2026 (ronda 3)

1. Resumen ejecutivo

Se realizó una auditoría dirigida del proyecto MediClick contra las 10 categorías del **OWASP Top 10 (edición 2021)**. De las categorías evaluadas, se identificaron y corrigieron hallazgos concretos en **A01 (Control de acceso roto)**, **A02 (Fallas criptográficas)**, **A06 (Componentes vulnerables)** y **A09 (Fallas de registro y monitoreo)**. El hallazgo más severo —ausencia de aislamiento multi-tenant en la API GraphQL, la única API autenticada del sistema— fue verificado a nivel de código y en tiempo de ejecución, y corregido con prueba de regresión.

El 9 de julio de 2026 se cerraron los dos pendientes puntuales que había dejado abiertos la ronda 1: el saneamiento de las dependencias directas vulnerables (A06) y el endpoint de consulta de la bitácora de seguridad (A09).

Corrección de alcance (9 jul 2026): la versión anterior de este informe marcaba A03, A05, A07 y A10 como "no auditado" sin distinguir entre "*nadie lo revisó*" y "*nadie lo revisó, pero el código ya mitiga el riesgo*". Una revisión de código dirigida (sección 6) encontró que estas cuatro categorías **ya tienen mitigación real implementada** (Prisma parametrizado + class-validator en A03; helmet/CORS/Swagger-off-en-prod en A05; JWT+bcrypt+rotación de refresh tokens+rate-limit en A07; único llamado saliente con host fijo y parámetro validado en A10) — simplemente nunca se auditó ni documentó como tal.

Sumando esas 4 a las que ya estaban reconocidas — 3 **CORREGIDAS** con auditoría formal y pruebas de regresión (A01, A02, A09) + 1 **PARCIAL** (A06) — el conteo real es: **8 de las 10 categorías del Top 10 tienen alguna mitigación en el código**. Solo A04 (Diseño inseguro) y A08 (Integridad de software/datos) siguen genuinamente sin abordar. De esas 8, la tabla distingue tres niveles de evidencia (ver también §9): 3

CORREGIDO

(auditadas + pruebas de regresión), 4

IMPLEMENTADO

(mitigación real en código, con

pruebas puntuales agregadas en §9.2, pero sin el proceso de auditoría completo) y 1 **PARCIAL** (A06: lo directo está resuelto, lo transitivo no).

Categoría	Nombre	Estado	Resultado
A01	Control de acceso roto (Broken Access Control)	CORREGIDO	Tenant scoping GraphQL + guard defense-in-depth en updateClinic + auditoría read-before-write
A02	Fallas criptográficas (Cryptographic Failures)	CORREGIDO	Cost factor de bcrypt elevado de 10 a 12
A03	Inyección (Injection)	IMPLEMENTADO	Prisma ORM (queries parametrizadas, sin <code>\$queryRawUnsafe</code> en toda la base de código) + <code>ValidationPipe</code> global con <code>whitelist</code> / <code>forbidNonWhitelisted</code> , con 3 pruebas automatizadas nuevas ACTUALIZADO
A04	Diseño inseguro (Insecure Design)	NO AUDITADO	Fuera de alcance; sin modelado de amenazas formal
A05	Configuración insegura (Security Misconfiguration)	IMPLEMENTADO	<code>helmet</code> + CORS restringido a <code>CLIENT_URL</code> + Swagger deshabilitado en producción + <code>trust proxy</code> correcto; sin auditoría formal de headers/infra ACTUALIZADO
A06	Componentes vulnerables (Vulnerable & Outdated Components)	PARCIAL	Auditoría en CI (informativa) + dependencias directas parcheadas (handlebars, nodemailer); deuda transitiva restante sin sanear
A07	Fallas de identificación y autenticación	IMPLEMENTADO	JWT+Passport, bcrypt cost 12, rotación de refresh tokens con detección de reuso (revoca todas las sesiones) y rate-limit de 5/min en login; sin política formal de expiración de sesión/fuerza de contraseña ACTUALIZADO
A08	Fallas de integridad de software y datos	NO AUDITADO	Fuera de alcance; sin integridad de pipeline CI/CD ni verificación de deserialización
A09	Fallas de registro y monitoreo (Logging & Monitoring)	CORREGIDO	Bitácora de auditoría append-only (login fallido + acceso denegado) + endpoint de consulta paginado

A10

Server-Side
Request Forgery
(SSRF)

IMPLEMENTADO

Único llamado HTTP saliente del backend (RENIEC) usa host fijo hardcodeado y parámetro validado por regex antes de interpolarse en la URL, con 6 pruebas automatizadas nuevas

ACTUALIZADO

2. A01 — Control de acceso roto (Broken Access Control)

2.1 Hallazgo principal: el aislamiento multi-tenant no se aplicaba en GraphQL

Severidad: Crítica

MediClick es multi-tenant: cada clínica es un tenant y los datos deben filtrarse por `clinicId`. El filtrado automático se implementa con una extensión de Prisma (`prisma.tenant`) que inyecta `clinicId` en las consultas, alimentada por un `TenantInterceptor` que lee el usuario autenticado del request.

El interceptor obtenía el request con `context.switchToHttp().getRequest()`. Esa llamada es válida en endpoints REST, pero en un *resolver* de GraphQL, `getArgByIndex(0)` retorna el **root del resolver**, no el request HTTP — por lo tanto `clinicId` quedaba `null` y la extensión de Prisma no aplicaba ningún filtro de tenant. Como GraphQL es la única API autenticada del sistema (REST solo expone el webhook público de MercadoPago y endpoints internos de reportes/auditoría con el mismo guard de permisos), el mecanismo de aislamiento no protegía efectivamente ninguna consulta real del producto.

Verificación: se reprodujo el `ExecutionContext` real que NestJS construye para resolvers GraphQL (`args = [root, args, {req}, info]`) y se confirmó en tiempo de ejecución que `switchToHttp().getRequest()` devolvía `undefined` /objeto vacío bajo ese contexto.

Corrección: se reemplazó la obtención del request por un helper existente (`getRequestFromContext`) que distingue el tipo de contexto y usa `GqlExecutionContext.create(context).getContext().req` cuando corresponde a GraphQL.

```
// server/src/shared/interceptors/tenant.interceptor.ts
const req = getRequestFromContext(context) as { user?: { clinicId?: number | null } };
const clinicId: number | null = req?.user?.clinicId ?? null;
```

Mitigación adicional ya existente (defensa en profundidad): las mutaciones de escritura por id ya seguían, en su mayoría, el patrón *read-before-write* (leer el recurso antes de escribir), lo que acotaba el impacto del bug a IDOR de **lectura** en queries — la escritura estaba parcialmente protegida incluso antes del fix.

2.2 Auditoría de read-before-write en escrituras por id

Se revisaron las ~30 mutaciones que escriben por id (`update` / `delete` / `upsert` sobre clave primaria) para verificar que ninguna permita modificar un recurso de otro tenant (IDOR de escritura). Conclusión: **el invariante se cumple** en todos los casos, mediante una de tres vías:

- **Lectura ya filtrada por tenant** (`prisma.tenant.X.findFirst`) — appointments, doctors, medical-history, prescriptions, specialties.

- **Lectura + verificación explícita** de `clinicId` en el caso de uso, lanzando `ForbiddenException` si no coincide — availability, categories, holidays, schedule-blocks, roles, users.
- **Verificación de pertenencia (ownership)** contra el paciente autenticado del token — lista de espera (cancelar / aceptar oferta).

2.3 Hallazgo secundario: updateClinic sin verificación de tenant

Severidad: Baja (riesgo latente, no explotable hoy)

La mutación que actualiza una sede (`UpdateClinicUseCase`) no verificaba que el solicitante solo pudiera modificar su propia sede. No era explotable en el estado actual del sistema porque únicamente el permiso `MANAGE:ALL` (rol de plataforma) tiene `UPDATE:CLINICS` asignado, y para ese rol el acceso cruzado entre sedes es intencional. Aun así, se corrigió como defensa en profundidad.

Por qué se discriminó por rol de sistema y no por clinicId: el usuario "admin de clínica" sembrado en el sistema usa el rol de plataforma ADMIN (equivalente a `MANAGE:ALL`) con un `clinicId` no nulo. Un guard ingenuo del tipo `if (clinicId && id !== clinicId)` habría bloqueado a ese super-administrador al editar una sede distinta a la suya. La solución correcta discrimina por el **rol de sistema** del solicitante (SUPER_ADMIN/ADMIN pueden administrar cualquier sede; el resto de roles, solo la propia).

```
// server/src/modules/clinics/application/use-cases/update-clinic.use-case.ts
const CROSS_TENANT_ROLES = new Set([SystemRole.SUPER_ADMIN, SystemRole.ADMIN]);

if (!CROSS_TENANT_ROLES.has(callerRoleName) && callerClinicId !== id) {
  throw new ForbiddenException('Solo puede modificar su propia sede');
}
```

2.4 Pruebas agregadas

- `tenant.interceptor.spec.ts` — 4 pruebas: reproduce el contexto GraphQL real, confirma que el bug original existía (regresión), confirma el comportamiento correcto en HTTP, y valida `clinicId` nulo para usuarios cross-tenant.
- `update-clinic.use-case.spec.ts` — 4 pruebas: deniega a rol acotado modificar otra sede, permite modificar la propia, permite a rol de plataforma modificar cualquiera, y `NotFound` antes de evaluar el tenant.

3. A02 — Fallas criptográficas (Cryptographic Failures)

El servicio centralizado de hashing de contraseñas (`PasswordService` , único punto de hashing en tiempo de ejecución del sistema) usaba un *cost factor* de bcrypt de `10` . Se elevó a `12` , dentro del rango recomendado por OWASP, para dar margen frente al avance de hardware sin impacto perceptible en el tiempo de login.

```
// server/src/modules/auth/infrastructure/services/password.service.ts
private readonly SALT_ROUNDS = 12; // antes: 10
```

Prueba agregada: se verifica que el cost factor real quede codificado en el hash resultante (prefijo `$2b$12$`) y se confirma el round-trip hash/compare. Los scripts de seed de datos de desarrollo (`prisma/seed.ts`), que usan una contraseña de prueba fija, se dejaron fuera de este cambio por no ser superficie de ataque real.

4. A06 — Componentes vulnerables y desactualizados

Se agregó un job `audit` al pipeline de CI (`.github/workflows/ci.yml`) que corre `pnpm audit --prod --audit-level high` sobre ambos workspaces (`server` y `client`) en cada push/PR a `main`.

Por qué quedó en modo informativo (`continue-on-error`): al ejecutar la auditoría se detectaron vulnerabilidades high/critical preexistentes en dependencias de producción (mayormente transitivas, arrastradas por Express, Prisma y Apollo). Activar el gate como bloqueante hoy habría dejado todos los PRs en rojo de forma inmediata. Se prioriza visibilidad continua sobre bloqueo prematuro; el gate debe subirse a obligatorio una vez saneada la deuda.

4.1 Actualización 9 de julio de 2026 — dependencias directas parcheadas

Cerrado (commit `882474d`): se evaluaron las dos dependencias directas señaladas como pendientes en la ronda 1.

- **handlebars** `^4.7.8 → ^4.7.9` — parche, bajo riesgo. Elimina 5 advisories high/critical (inyección JS por AST type confusion vía `@partial-block`, DoS por decorator syntax malformado, inyección en el CLI precompiler).
- **nodemailer** `^8.0.3 → ^9.0.3` — cambio mayor de versión, evaluado y aplicado. El único breaking change de la 9.0.0 es que las peticiones HTTPS para contenido remoto (attachments por URL, endpoints OAuth2, proxy CONNECT) ahora validan el certificado TLS por defecto. `MailService` de MediClick solo usa `createTransport / sendMail / verify` con SMTP básico, sin attachments remotos ni OAuth2, por lo que el cambio no afecta el comportamiento actual. Se actualizó también `@types/nodemailer` de `^7.0.11` a `^8.0.1` (DefinitelyTyped aún no publica una versión etiquetada para la serie 9.x, pero la superficie de API no cambió).

Resultado medido: `pnpm audit --prod --audit-level high` bajó de 20 a 14 advisories; severidad crítica de 1 a 0; alta de 19 a 14.

Deuda restante (transitiva, no atacada): `hono / @hono/node-server` (vía `prisma / @prisma/dev`), `multer` (vía `@nestjs/platform-express` y `@nest-lab/throttler-storage-redis`), `ws` (vía `@nestjs/graphql / Apollo`), `lodash`, `path-to-regexp`, `picomatch`, `defu`, `effect`. Ninguna es dependencia directa del proyecto; resolverlas requiere `pnpm.overrides` caso por caso, con riesgo real de incompatibilidad con las versiones de NestJS/Prisma/Apollo en uso. Se deja fuera de esta ronda a propósito — el gate de CI permanece informativo hasta evaluarlas una por una.

5. A09 — Fallas de registro y monitoreo de seguridad

El sistema no dejaba rastro de eventos de seguridad relevantes (intentos de login fallidos, accesos denegados por permisos). Se implementó una bitácora de auditoría dedicada:

- **Tabla `SecurityAuditLogs`** (append-only, sin claves foráneas de forma deliberada: el registro debe sobrevivir el borrado del usuario o la clínica que lo originó; `userId/email/clinicId` se guardan como snapshot del momento del evento).
- **`SecurityAuditService`** : registra eventos de forma *fire-and-forget* — un fallo al escribir la auditoría nunca convierte una respuesta 401/403 en un error 500 ni bloquea la petición del usuario.
- **Eventos cubiertos:** `LOGIN_FAILED` (capturado en `AuthController` al fallar la autenticación) y `PERMISSION_DENIED` (capturado en `PermissionsGuard` al denegar acceso por falta de permisos).

5.1 Actualización 9 de julio de 2026 — endpoint de consulta agregado

Cerrado (commit `cc7b24c`): la ronda 1 dejó la bitácora en solo-escritura, sin vista de lectura para el equipo de operaciones. Se agregó:

- `SecurityAuditService.findAllPaginated()` — consulta paginada con filtros opcionales por `eventType` y rango de fechas (`from` / `to`).
- `GET /security-audit-logs` en un `SecurityAuditController` nuevo, siguiendo el mismo patrón REST ya usado por el módulo de reportes (`@Auth()` + `@RequirePermissions('READ', 'SECURITY_AUDIT')`).
- Subject `SECURITY_AUDIT` agregado al catálogo de permisos sembrado (`prisma/seed.ts`) para poder asignarlo a roles granulares en el futuro. No fue necesario para que ADMIN accediera: `PermissionsGuard` ya trata `MANAGE:ALL` como wildcard total sobre cualquier acción/subject.

El alcance de los eventos capturados (solo `LOGIN_FAILED` y `PERMISSION_DENIED`) se mantiene igual que en la ronda 1, deliberadamente acotado para evitar ruido.

6. Categorías con mitigación implementada pero sin auditoría formal

Revisión de código dirigida (9 jul 2026) para responder una pregunta concreta: ¿cuántas categorías del Top 10 tienen ya mitigación real en el código, más allá de las 4 auditadas formalmente? Resultado: **A03, A05, A07 y A10 ya están implementadas**, solo que ninguna pasó por el proceso de auditoría con pruebas de regresión que sí tuvieron A01/A02/A09/A06.

6.1 A03 — Inyección: implementado

Sin ocurrencias de `$queryRaw` / `$executeRaw` en todo `server/src` — el 100% del acceso a datos pasa por el query builder tipado de Prisma (parametrizado por diseño, no hay concatenación de strings SQL en el proyecto). El pipe de validación global aplica *allow-listing* estricto de DTOs. Las opciones se extrajeron a una constante propia (commit `0c4fa16`) para poder probarlas sin duplicar valores:

```
// server/src/shared/config/security-validation-pipe.config.ts
export const SECURITY_VALIDATION_PIPE_OPTIONS: ValidationPipeOptions = {
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
};

// server/src/main.ts
app.useGlobalPipes(new ValidationPipe(SECURITY_VALIDATION_PIPE_OPTIONS));
```

`forbidNonWhitelisted` rechaza con 400 cualquier campo no declarado en el DTO — cierra la puerta a payloads con propiedades extra que muchos ataques de inyección/mass-assignment dependen.

Prueba automatizada (commit `0c4fa16`) — código real de `security-validation-pipe.config.spec.ts` :

```
class SampleDto {
  @IsString()
  @NotEmpty()
  name: string;
}

it('rechaza un payload con un campo no declarado en el DTO (mass-assignment)',
  async () => {
    await expect(
      pipe.transform({ name: 'Ana', roleId: 1 }, metadata),
    ).rejects.toBeInstanceOf(BadRequestException);
  });
```

pipe se construye en el `beforeEach` con `new ValidationPipe(SEcurity_VALIDATION_PIPE_OPTIONS)` — las mismas opciones exactas que corren en producción. La prueba manda `roleId`, un campo que `SampleDto` no declara, y confirma que el pipe lo rechaza con `BadRequestException` antes de que el valor llegue a cualquier lógica de negocio.

Falta: revisión caso por caso de otros vectores (logs, plantillas de `handlebars` en `TemplateService`, comandos del sistema).

6.2 A05 — Configuración insegura: implementado

```
// server/src/main.ts
app.use(helmet({
  contentSecurityPolicy: isProduction,
  crossOriginEmbedderPolicy: false,
}));
app.enableCors({ origin: process.env.CLIENT_URL ?? 'http://localhost:3000', credentials: true });
...
if (!isProduction) {
  // Swagger (/api/docs) solo se registra fuera de producción
}
```

Headers de seguridad vía `helmet`, CORS restringido a un origen conocido (no wildcard), y la documentación interactiva de la API (que expone el esquema completo) queda deshabilitada en producción. `trust proxy` está seteado explícitamente para que el rate-limit por IP no colapse detrás de Docker/nginx. **Falta:** auditoría de variables de entorno expuestas y configuración de Docker/infraestructura.

6.3 A07 — Identificación y autenticación: implementado

JWT + Passport, con `JWT_EXPIRES_IN` corto (15 min, `token.service.ts`) y refresh token de larga duración (7 días) en tabla separada. **Detección de reuso de refresh token** — si un refresh token ya

rotado se reintenta, se revocan *todas* las sesiones del usuario (mitigación estándar de OWASP contra robo de tokens):

```
// server/src/modules/auth/application/use-cases/refresh-token.use-case.ts
const incomingHash = this.tokenService.hashToken(rawRefreshToken);

// Reuse detection: token was already rotated → potential theft
if (incomingHash !== storedData.tokenHash) {
  this.logger.warn(
    `Refresh token reuse detected for user ${userId}, device ${deviceId}. Revoking all sessions.`
  );
  await this.refreshTokenRepository.deleteAllByUser(userId);
  throw new UnauthorizedException(
    'Se detectó reutilización de token. Todas las sesiones han sido cerradas por seguridad.'
  );
}
```

Prueba automatizada (ya existente) — `refresh-token.use-case.spec.ts`, **caso A07.3**: reproduce exactamente este escenario — llama `execute()` con un token cuyo hash no coincide con el guardado y verifica que se llame a `deleteAllByUser` y se lance `UnauthorizedException` con el mensaje de cierre de sesiones.

Rate-limit en endpoints de auth — actúa como freno de fuerza bruta aunque no hay bloqueo de cuenta formal:

```
// server/src/modules/auth/interfaces/controllers/auth.controller.ts
@Post('login')
@Throttle({ long: { ttl: 60000, limit: 5 } })
async login(@Body() dto: LoginDto, ...) { ... }
```

El mismo límite (5 intentos/min) se aplica también en registro y verificación de disponibilidad de email/documento. Sumado a bcrypt cost factor 12 (A02).

Falta: política explícita de fuerza de contraseña y bloqueo de cuenta tras N intentos (hoy solo hay rate-limit por IP/usuario, no un contador persistente de intentos fallidos); y una prueba de integración que dispare 6 requests y confirme el 429 (ver §9.3).

6.4 A10 — SSRF: implementado (por superficie mínima)

Todo `server/src` tiene **un solo** punto donde el backend hace una petición HTTP saliente con datos influenciados por el usuario: la consulta a RENIEC.

```
// server/src/modules/auth/application/use-cases/lookup-document.use-case.ts
if (!/^\d{8}$/.test(dto.numberDocument)) { return { found: false }; }
...
const url = `https://api.decolecta.com/v1/reniec/dni?numero=${dto.numberDocument}`;
const response = await fetch(url, { headers: {...}, signal: AbortSignal.timeout(5000) });
```

El host está hardcodedo (no es configurable ni viene del usuario) y el único parámetro interpolado se valida contra `/^\d{8}$/` antes de construirse la URL — no hay forma de redirigir la petición a un host distinto ni de inyectar un path. El resto de integraciones salientes (MercadoPago, SMTP) usan SDKs/hosts fijos, no URLs arbitrarias.

Prueba automatizada (commit `0c4fa16`) — código real de `lookup-document.use-case.spec.ts` :

```
it('rechaza un intento de inyectar un host/path distinto vía numberDocument', async
() => {
  const result = await useCase.execute({
    typeDocument: 'DNI',
    numberDocument: '12345678.evil.com/steal',
  });

  expect(result).toEqual({ found: false });
  expect(fetchSpy).not.toHaveBeenCalled();
});

it('con DNI valido, llama SIEMPRE al mismo host fijo con el numero exacto', async
() => {
  ...
  expect(fetchSpy).toHaveBeenCalledWith(
    'https://api.decolecta.com/v1/reniec/dni?numero=12345678',
    expect.objectContaining({ headers: { ... } })),
  );
});
```

`fetchSpy` es un `jest.spyOn(global, 'fetch')` — la prueba no llama a la red real, verifica que el código de la aplicación nunca intente construir una URL con el payload malicioso. En total 6 pruebas: la de arriba, DNI de 3 dígitos, tipo de documento distinto a DNI, sin `RENIEC_API_TOKEN` configurado, la llamada válida mostrada arriba, y un 500 de la API externa manejado sin lanzar.

Falta: revisar si `MailService` `/attachments` algún día aceptan URLs de usuario (hoy no lo hacen).

7. Categorías genuinamente sin abordar

- **A04 — Diseño inseguro:** sin modelado de amenazas formal. (El proyecto sí tiene decisiones de diseño seguro puntuales — p.ej. la discriminación por rol de sistema en A01 §2.3 — pero no como proceso sistemático.)
- **A08 — Fallas de integridad de software y datos:** sin firma/verificación de artefactos de CI/CD, sin revisión de deserialización de datos no confiables.

8. Commits y trazabilidad

Commit	Categoría	Descripción
<code>e59d67f</code>	A01	Tenant scoping en GraphQL (no solo REST)
<code>cdbcd4b</code>	A09	Bitácora de auditoría de eventos de seguridad

b9a8f95	A02	Cost factor de bcrypt de 10 a 12
247b27f	A06	Auditoría de dependencias en CI
5ecc615	A01	Guard de tenant en updateClinic (defense-in-depth)
90d25c5	—	Limpieza de lint introducido por los cambios de seguridad
e754494	—	Merge a main vía PR #35 (cierre ronda 1)
882474d	A06	Bump handlebars/nodemailer — cierre de pendiente (ronda 2)
cc7b24c	A09	Endpoint de consulta de la bitácora — cierre de pendiente (ronda 2)
6c91c8d	—	Documentar A03/A05/A07/A10 como implementados (corrección de alcance)
0c4fa16	A03, A10	Pruebas automatizadas nuevas — cierre de la brecha de evidencia (ronda 3)

9. Verificación y evidencia de pruebas

9.1 Cómo reproducir

```
cd server
pnpm install
npx tsc --noEmit          # 0 errores
npx jest --silent         # 50 suites / 341 pruebas en verde
pnpm audit --prod --audit-level high # 14 advisories, 0 criticas
```

Resultado actual (9 jul 2026, ronda 3): 50 suites / 341 pruebas, todas en verde. `tsc --noEmit` : 0 errores. Lint limpio en todos los archivos agregados/modificados de las 3 rondas. Migración de base de datos de `SecurityAuditLogs` aplicada y verificada con inserción/borrado real (ronda 1).

9.2 Qué prueba cada suite — matriz de evidencia por categoría

Para cada categoría marcada CORREGIDO o IMPLEMENTADO, esta tabla enumera el archivo de test exacto, cuántas pruebas tiene y qué escenario de seguridad demuestra cada una — no solo "hay tests", sino qué comportamiento específico verifican.

Categoría	Archivo	#	Qué demuestra
A01	tenant.interceptor.spec.ts	4	Reproduce el <code>ExecutionContext</code> real de GraphQL y confirma que el bug (<code>clinicId</code> null) existía; confirma que en HTTP/REST sí funcionaba; confirma el fix en GraphQL; valida <code>clinicId</code> null para roles cross-tenant (paciente/super-admin).
A01	update-clinic.use-case.spec.ts	4	Deniega a un rol clínico-scopeado modificar OTRA sede; permite modificar la PROPIA; permite a rol de plataforma modificar cualquiera; lanza 404

antes de evaluar tenant (no filtra existencia de sedes ajenas).

A01 / A09	<code>permissions.guard.spec.ts</code>	13	Matriz completa de RBAC: sin <code>@RequirePermissions</code> permite; sin usuario/sin roleId deniega; permiso exacto concede; permiso faltante deniega; ningún permiso deniega; <code>MANAGE:ALL</code> permite todo; <code>MANAGE:{subject}</code> permite ese recurso; un wildcard de subject erróneo NO cuela; caché Redis hit/miss; registra <code>PERMISSION_DENIED</code> al denegar y NO registra nada al conceder.
A02	<code>password.service.spec.ts</code>	3	El cost factor 12 queda codificado en el prefijo del hash (<code>\$2b\$12\$</code>); un hash compara verdadero contra su password original; compara falso contra una password distinta.
A03	<code>security-validation-pipe.config.spec.ts</code> NUEVO	3	Acepta un payload con solo los campos declarados en el DTO; rechaza un payload con un campo no declarado (mass-assignment); rechaza un payload que no cumple las validaciones — con las mismas opciones literales de <code>main.ts</code> .
A06	(no aplica jest)	—	Verificado por herramienta externa: <code>pnpm audit --prod --audit-level high</code> , no por un test unitario. Ver §9.1 para el número exacto.
A07	<code>refresh-token.use-case.spec.ts</code>	8	Rota el refresh token correctamente (A07.1); rechaza si no existe token para el dispositivo (A07.2); detecta reutilización del token y revoca TODAS las sesiones (A07.3, x2); rechaza usuario desactivado/eliminado y revoca sus sesiones (A07.4, x3); Redis caído → 503, nunca 500 (A07.5).
A09	<code>security-audit.service.spec.ts</code>	4	Persiste el evento con campos normalizados; nunca lanza si falla la escritura (no convierte 401/403 en 500); el query paginado filtra por <code>eventType</code> y rango de fechas; consulta sin filtros pagina la bitácora completa.
A10	<code>lookup-document.use-case.spec.ts</code> NUEVO	6	Rechaza DNI mal formado sin llamar <code>fetch</code> ; rechaza un intento de inyectar host/path distinto vía el parámetro ; rechaza tipos de documento distintos a DNI; rechaza sin token configurado; una llamada válida usa siempre el mismo host fijo con el número exacto; un error de la API externa retorna <code>found:false</code> sin lanzar.

9.3 Lo que NO tiene prueba automatizada (honestidad de alcance)

- **A05 (helmet/CORS/Swagger-off-en-prod):** es configuración de bootstrap en `main.ts`, verificada hoy solo por inspección de código (líneas citadas en §6.2), no por un test que levante la app y revise headers de respuesta. El proyecto no tiene arnés e2e: `package.json` referencia `test:e2e` → `jest --config ./test/jest-e2e.json`, pero ese archivo y el directorio `test/` no existen — es boilerplate de Nest nunca completado. Montar un test e2e real (supertest + app completa) es la forma correcta de cerrar esta brecha, pero es un esfuerzo aparte (requiere decidir estrategia de base de datos de test).
- **A06 (dependencias):** la evidencia es la salida de `pnpm audit` (herramienta externa), no una prueba de Jest — no hay (ni tiene sentido escribir) un test unitario que verifique "esta versión de un paquete no es vulnerable".
- **A07, rate-limit (@Throttle):** el decorador está presente en `auth.controller.ts` con los límites citados (verificable leyendo el archivo), pero no hay un test que dispare 6 requests y confirme el 429 — `@nestjs/throttler` es una librería externa ya probada por su propio equipo; probar la integración requeriría el mismo arnés e2e mencionado arriba.

10. Hallazgos pendientes y deuda conocida

- **A06:** ~14 vulnerabilidades high transitivas en dependencias de producción (hono, multer, ws, lodash, path-to-regexp, picomatch, defu, effect), sin sanear; requieren `pnpm.overrides` evaluados caso por caso por riesgo de incompatibilidad. El gate de CI permanece informativo hasta resolverlas.
- **A05:** implementada (§6.2) pero sin prueba automatizada ni auditoría de infraestructura/Docker — requiere montar el arnés e2e que hoy no existe en el proyecto (§9.3).
- **A07:** implementada y con 8 pruebas automatizadas (§9.2), pero falta política formal de bloqueo de cuenta/fuerza de contraseña, y el rate-limit (`@Throttle`) no tiene prueba de integración (§9.3).
- **A03, A10:** implementadas y ahora con pruebas automatizadas nuevas (§9.2) — falta validación caso por caso de otros vectores en A03, y confirmar con auditoría formal que ninguna otra ruta del código agregue una segunda llamada saliente en A10.
- **A04, A08:** sin mitigación dirigida (sección 7) — pendientes de una ronda de auditoría desde cero.
- **Deuda de calidad de código (no es un hallazgo OWASP):** el pipeline de CI tiene el paso de `lint` en rojo de forma crónica (~1740 problemas preexistentes de `@typescript-eslint/no-unsafe-*` en todo el código base, no introducidos por este trabajo). Sanear el resto es un esfuerzo independiente.

Resueltos en esta actualización (9 jul 2026): saneamiento de dependencias directas vulnerables de A06 (handlebars, nodemailer), endpoint de consulta de la bitácora de A09, corrección de alcance documentando que A03/A05/A07/A10 ya tenían mitigación real implementada (8/10 categorías con alguna protección en el código: 3 CORREGIDO + 4 IMPLEMENTADO + 1 PARCIAL), y cierre de la brecha de evidencia para A03 y A10 con 9 pruebas automatizadas nuevas (antes verificadas solo por inspección de código).

`cc7b24c`), corrección de alcance para A03/A05/A07/A10 (commit `6c91c8d`) y pruebas automatizadas nuevas para A03/A10 (commit `0c4fa16`). Referencia: OWASP Top 10:2021 — owasp.org/Top10.